

# Survey Instrument: MedReport Interpreter Maintainability Evaluation

## Case Study:

MedReport Interpreter is an **AI-based web application** that helps people understand their medical lab reports.

- The patient uploads a **PDF or image** of their lab report
- The system uses **OCR** to read the report
- Then AI processes the data
- It gives **two outputs**:
  1. **Simple summary** (easy for patients)
  2. **Clinical summary** (for doctors)

It also:

- Compares **current and past reports** to show health trends
- Helps reduce confusion, stress, and wrong understanding of reports

## Maintainability Characteristics Mapping

### 1. Retrainingability

- Medical knowledge changes over time
- The system can be **updated and retrained** with new data

This means the system stays **up-to-date**

### 2. Reproducibility

- If a result is wrong, it should be checked
- The system **stores all details** (input file, model version, output)

This means results can be **tracked and verified**

### 3. Explainability

- Patients need to understand their results
- The system explains:
  - Which test is abnormal
  - Normal range
  - What it means

This means results are **clear and understandable**

### 4. Monitorability

- AI performance can decrease over time
- The system is **continuously monitored**:
  - Accuracy is checked
  - Errors are tracked

This means the system stays **reliable and accurate**

You will evaluate the OOP class diagram of MedReport Interpreter against 4 AI maintainability characteristics: Retrainability, Reproducibility, Explainability, and Monitorability. Please review the attached class diagram image before answering."

**Domain Model Link:** [Domain Model \(class diagram\)](#)

### Study Information

This system has 5 modules (M1–M5) and 13 classes. Evaluation takes approx. 10–14 minutes.

### Consent\*

I have reviewed the MedReport Interpreter class diagram and understand the purpose of the system before proceeding.

### Current role

Software engineering student (UG)

Software engineering student (MS/PhD)

Software engineer / developer

Software architect

Academic / lecturer

AI / ML engineer

Healthcare IT professional

Other:

### Years working with OOP / UML design

Less than 1 year

1–3 years

4–7 years

8+ years

### Familiarity with AI system quality / maintainability

None — first time

Basic awareness

Working knowledge

Expert level

### Exposure to healthcare / medical software systems

None

Academic / coursework only

Some industry exposure

Significant experience

### Section 1 of 5 — M5: Model Retraining & Versioning

Classes: ModelManager, MonitoringService. ModelManager handles lifecycle (retrain, validate, rollback, deploy).

#### M5 Evaluation: Retraining & Monitorability

1 (Strongly Disagree)

2

3

4

5 (Strongly Agree)

Q1. The separation of ModelManager and MonitoringService supports safe, independent retraining.

Q2. The rollback(version) method is sufficient to recover from failed deployment.

Q3. MonitoringService tracking methods provide adequate coverage to detect degradation.

Q4. The alertDegradation() method triggering ModelManager is a clean design.

Q1. The separation of ModelManager and MonitoringService supports safe, independent retraining.

Q2. The rollback(version) method is sufficient to recover from failed deployment.

Q3. MonitoringService tracking methods provide adequate coverage to detect degradation.

Q4. The alertDegradation() method triggering ModelManager is a clean design.

Q5. What additional attribute or method, if any, would strengthen M5's retrainability or monitorability?

### Section 2 of 5 — M4: User Management & Logging

#### M4 Evaluation: Reproducibility

1 (Strongly Disagree)

2

3

4

5 (Strongly Agree)

Q6. AuditLogger captures sufficient info to reconstruct past interpretation sessions.

Q7. Storing fileHash is necessary for verifying input integrity.

Q8. AuditLogger design as a separate class is correct for independent audit concerns.

Q6. AuditLogger captures sufficient info to reconstruct past interpretation sessions.

Q7. Storing fileHash is necessary for verifying input integrity.

Q8. AuditLogger design as a separate class is correct for independent audit concerns.

Q9. Is there any data that AuditLogger is currently NOT storing that would be critical?

### Section 3 of 5 — M2: AI Interpretation Engine

#### M2 Evaluation: Explainability & Reproducibility

1 (Strongly Disagree)

2

3

4

5 (Strongly Agree)

Q10. explainAbnormality() is correctly placed and has the right parameters.

Q11. RAGEngine.groundResponse() meaningfully supports explainability.

Q12. Using subclasses (Layman/Clinical) is correct for different audiences.

Q13. sessionId/modelVersion + AuditLogger are sufficient for reproduction.

Q10. explainAbnormality() is correctly placed and has the right parameters.

Q11. RAGEngine.groundResponse() meaningfully supports explainability.

Q12. Using subclasses (Layman/Clinical) is correct for different audiences.

Q13. sessionId/modelVersion + AuditLogger are sufficient for reproduction.

Q14. Is there an explainability concern in this module that the design does NOT address?

### Section 4 of 5 — M1 & M3 (OCR & Trends)

#### M1 & M3 Evaluation

1 (Strongly Disagree)

2

3

4

5 (Strongly Agree)

- Q15. OCR confidence attribute is a meaningful signal for quality degradation.
- Q16. Redundant fileHash storage is justified for verification.
- Q17. Trend classification labels (Stable/Recovering) contribute to explainability.
- Q18. Independent design of HistoricalAnalyzer from the AI model is correct.
- Q15. OCR confidence attribute is a meaningful signal for quality degradation.
- Q16. Redundant fileHash storage is justified for verification.
- Q17. Trend classification labels (Stable/Recovering) contribute to explainability.
- Q18. Independent design of HistoricalAnalyzer from the AI model is correct.

## Section 5 of 5 — Overall System Evaluation

### Overall Characteristic Assessment

1 (Strongly Disagree)

2

3

4

5 (Strongly Agree)

- Q19. Retraining: Supports model retraining as a routine, low-risk operation.
- Q20. Reproducibility: Traces every output via AuditLogger and fileHash.
- Q21. Explainability: Delivers appropriate explainability for medical AI.
- Q22. Monitorability: Design is sufficient to prevent silent AI degradation.
- Q19. Retraining: Supports model retraining as a routine, low-risk operation.
- Q20. Reproducibility: Traces every output via AuditLogger and fileHash.
- Q21. Explainability: Delivers appropriate explainability for medical AI.
- Q22. Monitorability: Design is sufficient to prevent silent AI degradation.

Q23. Which characteristic is LEAST well-supported by the current class design?

Retrainability

Reproducibility

Explainability

Monitorability

Q24. Would you use this class diagram as a reference OOP design for building a similar system?

Yes, as-is

Yes, with modifications

No

Q25. Suggest one specific change to the class diagram that would most improve the system.